

FICHA

Nathaly Campos - C.I. 32.460.314

15 de mayo de 2026

1. Lenguaje ensamblador MIPS -

Resumen arquitectura MIPS.

Categoría	Instrucción	Ejemplo	Significado	Comentarios
Aritmética	add	add \$s1,\$s2,\$s3	$\$s1 = \$s2 + \$s3$	Tres operandos; datos en registros
	add immediate	addi \$s1,\$s2,100	$\$s1 = \$s2 + 100$	Usado para sumar constantes
Transferencia de dato	load word	lw \$s1,100(\$s2)	$\$s1 = \text{Memory}[\$s2 + 100]$	Palabra de memoria a registro
	store word	sw \$s1,100(\$s2)	$\text{Memory}[\$s2 + 100] = \$s1$	Palabra de registro a memoria
	store half	sh \$s1,100(\$s2)	$\text{Memory}[\$s2 + 100] = \$s1$	Media palabra de registro a memoria
Lógica	shift left logical	sll \$s1,\$s2,10	$\$s1 = \$s2 \ll 10$	Desplazamiento a la izquierda por constante
Salto condicional	branch on equal	beq \$s1,\$s2,L	if ($\$s1 == \$s2$) go to L	Comprueba igualdad y bifurca relativo al PC
	branch on not equal	bne \$s1,\$s2,L	if ($\$s1 \neq \$s2$) go to L	Comprueba si no igual y bifurca relativo al PC
Salto incond.	jump	j 2500	go to 10000	Salto a la dirección destino

2. Explicación de comandos

- **add (Suma):** Es tu suma tradicional. Tomas dos variables (registros), las sumas y guardas el resultado en una tercera. *Ej: add \$t0, \$t1, \$t2 ($t0 = t1 + t2$).*
- **addi (Suma inmediata):** Igual que la anterior, pero en lugar de sumar dos variables, le sumas un número fijo (una constante) a una variable. *Ej: addi \$t0, \$t1, 5 ($t0 = t1 + 5$).*
- **lw (Load Word - Cargar palabra):** Es el equivalente a "leer". Vas a la memoria principal (RAM), agarras un dato completo de 32 bits y te lo traes al procesador para poder trabajar con él. *Ej: lw \$t0, 0(\$s0).*
- **sw (Store Word - Guardar palabra):** Es el equivalente a "escribir". Tomas un dato de 32 bits que ya procesaste en tus registros y lo mandas a guardarse en la memoria principal. *Ej: sw \$t0, 0(\$s0).*
- **sh (Store Half - Guardar media palabra):** Hace lo mismo que **sw**, pero solo guarda la mitad del dato (16 bits). Es útil cuando quieres ahorrar espacio y sabes que tu número no es tan grande. *Ej: sh \$t0, 0(\$s0).*

- **sll (Shift Left Logical - Desplazamiento):** Mueve todos los bits de un número hacia la izquierda. En la práctica, es un truco rápido para multiplicar por potencias de 2 (desplazar 1 vez es multiplicar por 2, desplazar 2 veces es multiplicar por 4, etc.). *Ej: `sll $t0, $t1, 2`.*
- **beq (Branch on Equal - Salto si es igual):** Es tu condicional `if (a == b)`. Compara dos registros; si son exactamente iguales, salta a otra línea de código. *Ej: `beq $t0, $t1, Etiqueta`.*
- **bne (Branch on Not Equal - Salto si no es igual):** Es el opuesto, tu `if (a != b)`. Si los registros son diferentes, salta. *Ej: `bne $t0, $t1, Etiqueta`.*
- **j (Jump - Salto incondicional):** El clásico `goto`. Obliga al programa a ir a una línea específica sin preguntar nada. Se usa mucho para reiniciar bucles. *Ej: `j Etiqueta`.*

3. Operandos MIPS: Los 32 Registros

- **\$zero (Reg 0):** Siempre es igual a 0 por diseño de hardware. Es útil para inicializar otros registros o hacer comparaciones rápidas.
- **\$at (Reg 1):** Temporal del ensamblador. Está reservado por el sistema para manejar instrucciones complejas o constantes grandes. No se debe usar para programar.
- **\$v0 - \$v1 (Reg 2-3):** Valores de retorno. Se usan para devolver los resultados finales calculados tras llamar a una subrutina o función.
- **\$a0 - \$a3 (Reg 4-7):** Argumentos. Son la puerta de entrada; se utilizan para pasarle datos y parámetros a una subrutina antes de ejecutarla.
- **\$t0 - \$t9 (Reg 8-15, 24-25):** Temporales. Variables rápidas para cálculos intermedios. Su valor no está garantizado si el programa salta a otra función.
- **\$s0 - \$s7 (Reg 16-23):** Seguros (Saved). Variables a largo plazo. Su valor sobrevive intacto al llamar a otras subrutinas.
- **\$gp (Reg 28):** Puntero global. Referencia para acceder rápidamente a la zona de memoria donde están las variables estáticas y globales.
- **\$sp (Reg 29):** Puntero de pila (Stack Pointer). Apunta a la última posición ocupada en la memoria temporal (pila), usado para `empujar` o `sacar` datos.
- **\$fp (Reg 30):** Puntero de marco (Frame Pointer). Sirve como referencia base fija para encontrar las variables locales dentro de una función activa.
- **\$ra (Reg 31):** Dirección de retorno. Al saltar a una función, guarda automáticamente la dirección de la línea de código a la que se debe regresar al terminar.